

Electrical and Computer Engineering Department
School of Engineering and Computer Science
Oakland University
Automotive Mechatronics I
Summer 2025

Course Project

Adaptive Cruise Control (ACC) and Forward Collision Avoidance (FCA) using Controller Area
Network (CAN)

Students Names: Dale Szydlowski, Eric Jannowiecki

Instructor Name: Nashwan J. Sebi

Sending Side (TX) Code:

```

ECE_4520_Course_Project_Transmit_Side.ino | Arduino IDE 2.3.6
File Edit Sketch Tools Help

Arduino UNO R4 Minima

ECE_4520_Course_Project_Transmit_Side.ino
2  #include <mcp_can.h>
3  #include <SPI.h>
4  #ifndef ARDUINO_SAMD_VARIANT_COMPLIANCE
5  | #define SERIAL SerialUSB
6  #else
7  | #define SERIAL Serial
8  #endif
9  const int SPI_CS_PIN = 10;
10 MCP_CAN CAN(SPI_CS_PIN);
11 const int bufferSize = 40;
12 int buffer[bufferSize];
13 int index;
14 int IRin=0;
15 unsigned char distance [8];
16 void setup()
17 {
18   pinMode (IRin, INPUT);
19   Serial.begin (9600);
20   while (CAN_OK != CAN.begin(CAN_500KBPS))
21   {
22     Serial.println("CAN BUS Shield init fail");
23     Serial.println(" Init CAN BUS Shield again");
24     delay (100);
25   }
26   Serial.println("CAN BUS Shield init ok!");
27 }
28 void loop ()
29 {
30   int a=analogRead(IRin);
31   Serial.print(a); Serial.print ("\\t");
32   int Range = ((6787/(a +3))-4);
33   Serial.println(average());
34   buffer[index] = Range;
35
36   index++;
37   if (index >= bufferSize) index = 0;
38   distance [7]= average ();
39   CAN.sendMsgBuf (0x07, 0, 8, distance);
40   delay (10);
41 }
42 int average ()
43 {
44   long sum = 0;
45   for (int i = 0; i < bufferSize; i++)
46   {
47     sum += buffer[i];
48   }
49   return (int) (sum / bufferSize);
50 }

```

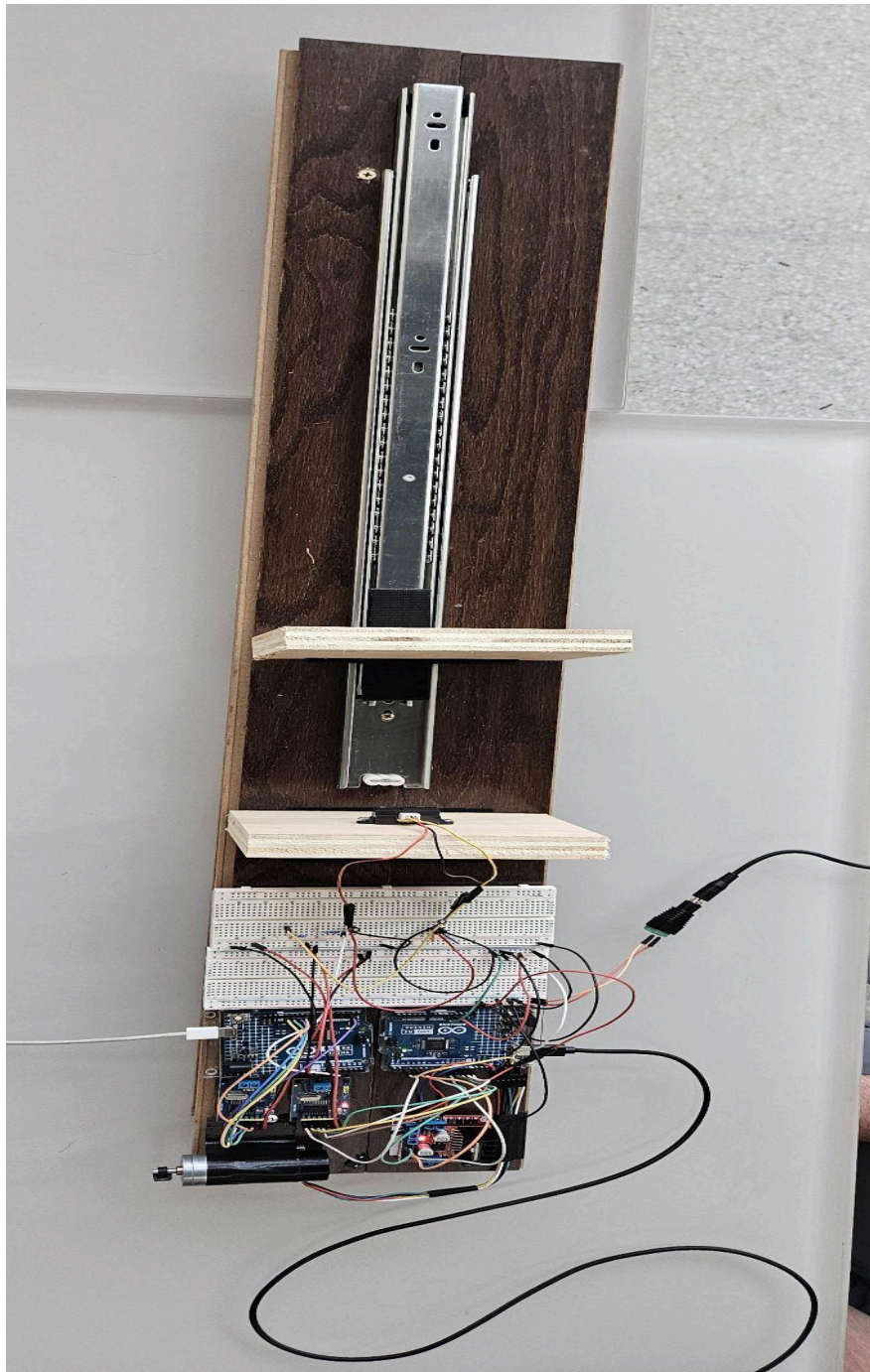
Receiving Side (RX) Code:

```

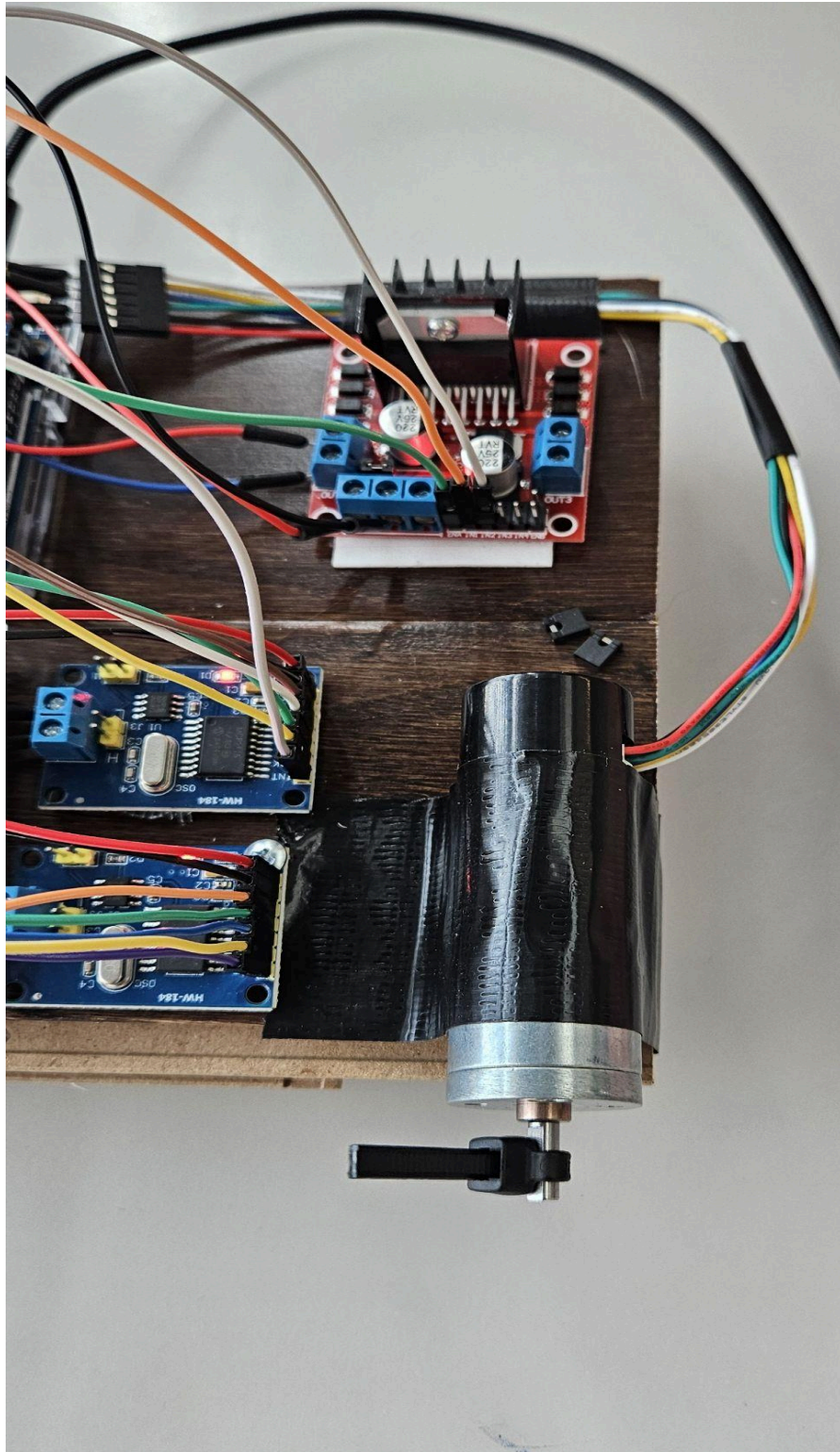
ECE_4520_Course_Project_Receiv_Side.ino
1 // speed control of geared DC motor with encoder-(CAN-recieve)-- Nashwan J. Sebi -- Dec. 29th, 2019
2 #define ENCODEROUTPUT 540 // PPR pulse per rotation
3 #define PWM 9 // L298 PWM connected to pin 9
4 #define encoderPinA 3 // Hall sensor A connected to pin 3 (external interrupt)
5
6 #include <mcp_can.h>
7
8 unsigned char buf[8];
9 const int SPI_CS_PIN = 10; // Set CS pin for CAN module
10 MCP_CAN CAN (SPI_CS_PIN);
11
12 //driving one way motor encoder
13 int M_left =4; //direction IN1 (L298N)
14 int E_left =5; //direction IN2 (L295N)
15 int rpm = 0;
16 volatile unsigned long encoderValue = 0; // pulse count
17 unsigned long lastmillis = 0;
18
19 void setup ()
20 {
21   Serial.begin (9600);
22   pinMode (M_left, OUTPUT); //L298N IN1
23   pinMode (E_left, OUTPUT); //L298N IN2
24   pinMode (encoderPinA, INPUT_PULLUP); // Set hall sensor A as input pullup
25   pinMode (PWM, OUTPUT); // Set PWM pin as output
26   digitalWrite (M_left, LOW); // CW direction
27   digitalWrite (E_left, HIGH); // CW direction
28   attachInterrupt(digitalPinToInterrupt(encoderPinA), updateEncoder, FALLING);
29
30   while (CAN_OK != CAN.begin(CAN_500KBPS))
31   {
32     Serial.println("CAN BUS Shield init fail");
33
34     Serial.println(" Init CAN BUS Shield again");
35     delay (100);
36   }
37   Serial.println("CAN BUS Shield init ok!");
38 }
39
40 void loop()
41 {
42   unsigned char len = 0;
43   float v;
44   float pi=3.14159265;
45
46   if (CAN_MSGAVAIL == CAN.checkReceive()) // if something received
47   {
48     CAN.readMsgBuf(&len, buf); // read buf
49     unsigned long canid = CAN.getCanId(); // get can id when receive
50     Serial.println("-----");
51     int value = (int)buf[7];
52
53     if (value >= 12 && value <= 25) {
54       int val_output = map(value, 12, 25, 0, 255);
55
56       while (millis() - lastmillis != 1000)
57       { //Uptade every one second
58         analogWrite (PWM, val_output) ;
59       }
60
61       detachInterrupt(digitalPinToInterrupt(encoderPinA)); //Disable interrupt when calculating
62       rpm = (float) ( encoderValue * 60/ ENCODEROUTPUT) ;
63       Serial.print ("rpm =");
64       Serial.println(rpm);
65       Serial.print ("Linear Velocity (V)=");Serial.print(v=((2*pi/60)*rpm*0.03));
66
67       Serial.println ("m/s");
68       Serial.print ("PWM =");
69       Serial.println(val_output);
70       Serial.print ("dis =");
71       Serial.println (value);
72       encoderValue = 0;
73       lastmillis = millis();
74       attachInterrupt(digitalPinToInterrupt(encoderPinA), updateEncoder, FALLING);
75     }
76     else Stop();
77   }
78
79 void updateEncoder ()
80 {
81   encoderValue++; // Add encoderValue by 1, each time it detects rising signal from hall sensor A
82 }
83 void Stop()//Motor stops
84 {
85   digitalWrite(PWM, LOW);
86 }

```

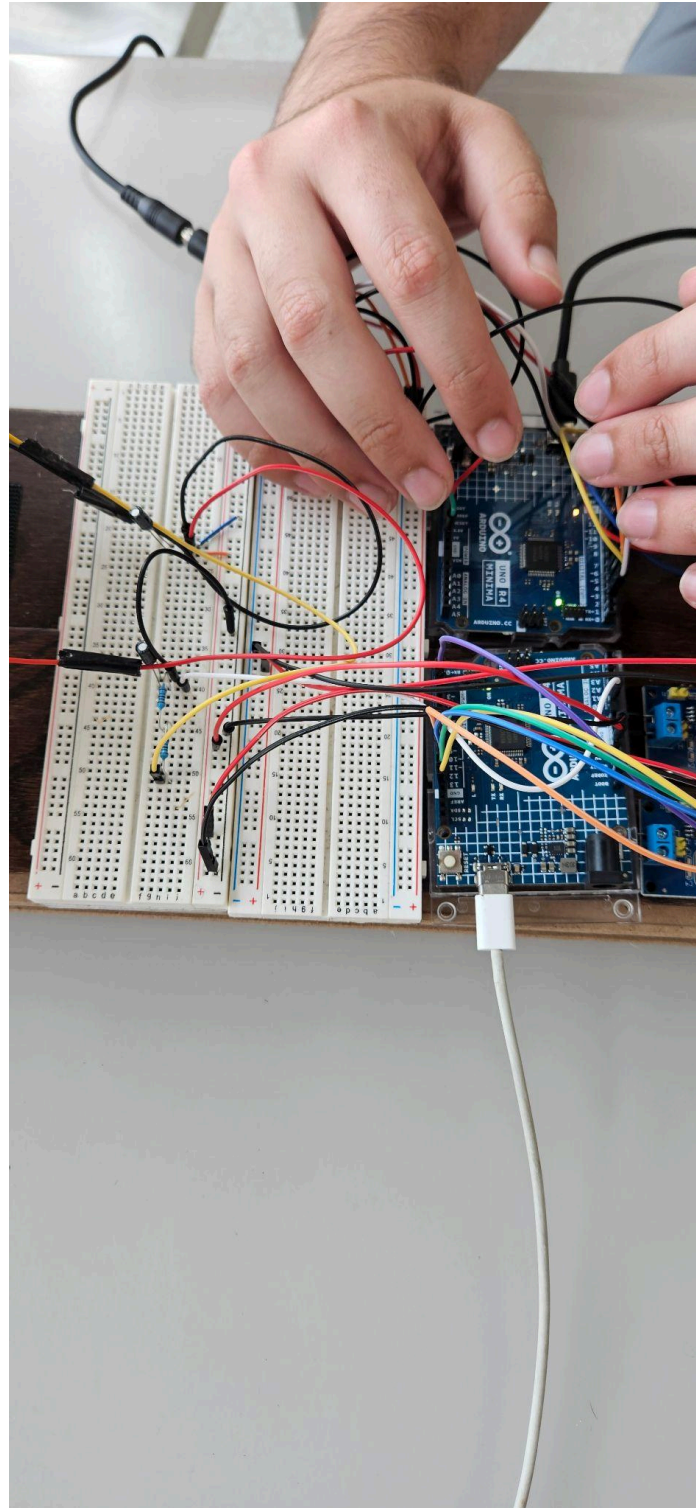
Project Photos:



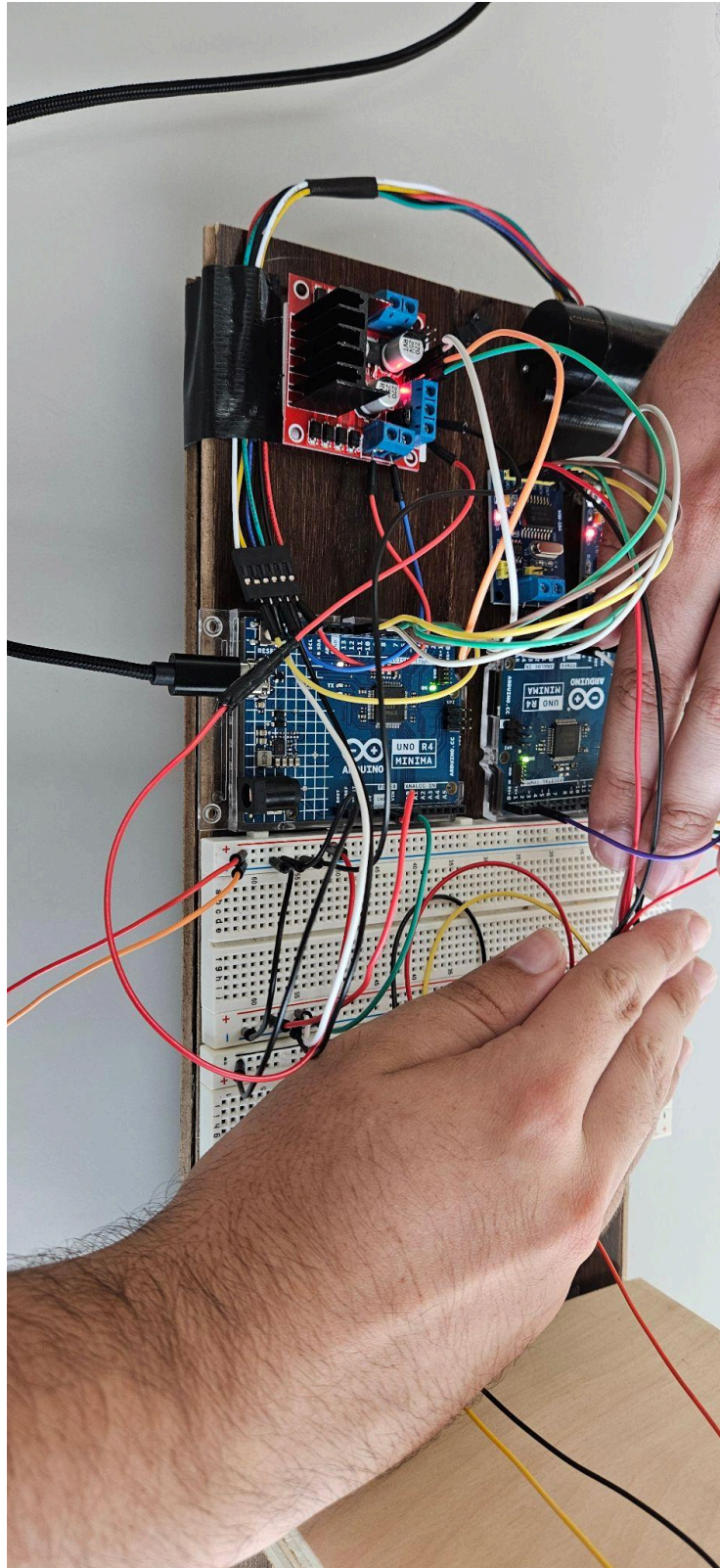
Picture of Project



Picture of Motor



Picture of Sending Side (TX)



Picture of Receiving side (RX)

Data Values:

```
-----  
rpm =0.00  
Linear Velocity (V)=0.00m/s  
PWM =19  
dis =13  
-----
```

Output Value at 13cm
(RPM is zero due to low power provided)

```
-----  
rpm =31.11  
Linear Velocity (V)=0.10m/s  
PWM =58  
dis =15  
-----
```

Output Value at 15cm

```
-----  
rpm =82.44  
Linear Velocity (V)=0.26m/s  
PWM =137  
dis =19  
-----
```

Output Value at 19cm

```
-----  
rpm =101.67  
Linear Velocity (V)=0.32m/s  
PWM =215  
dis =23  
-----
```

Output Value at 23cm

Questions:

- 1) Did you need both of the DC motor's outputs (A and B) pulses to calculate the RPM?

We did not need both of the motor's output signals to calculate the RPM.

- 2) If your answer was you needed only one output, A or B, to calculate the RPM. Then why do you think that the quadrature encoder has two outputs? In other words, what are they useful for?

We believe that the encoder has two outputs for redundancy purposes. Having two processors monitor the output value of the motor can allow for proper operation of the system even if one encoder/microprocessor fails in its operation.